

**FIAP – FACULDADE DE INFORMÁTICA E
ADMINISTRAÇÃO PAULISTA**

PROMPT ENGINEERING & ARTIFICIAL INTELLIGENCE

GRUPO NEXTGEN

Gustavo Franzoti Gonçalves – RM 566983
Lincoln Simão Pereira – RM 567284
Maykon Santana Fonseca – RM 567041
Nicolas Sakaue Nishimura – RM 567752

CHECKPOINT 02 – PROMPT TOOLKIT

**Toolkit em Python para aplicação automática de técnicas de
prompting com avaliação comparativa**

Documentação técnica apresentada como requisito parcial para avaliação do Checkpoint 02 da disciplina de Prompt Engineering & Artificial Intelligence – Módulo 2, sob orientação do Prof. Jorge Luiz Gomes.

São Paulo
2026

SUMÁRIO

1	Introdução	2
2	Domínio e Problema	2
2.1	Domínio escolhido: <i>E-commerce</i> e atendimento ao cliente	2
2.2	Problema real	2
2.3	Justificativa da escolha	3
3	Arquitetura do Projeto	3
3.1	Estrutura de pastas	3
3.2	Diagrama do fluxo de execução	3
3.3	Responsabilidades dos módulos	4
4	Stack Técnica	4
4.1	Tecnologias utilizadas	4
4.2	Dependências principais (<code>requirements.txt</code>)	5
4.3	Como executar	5
5	Técnicas e Tarefas Implementadas	6
5.1	As quatro técnicas de prompting	6
5.2	Personas detalhadas (Aula 07)	6
5.3	Tarefas do domínio	6
6	Resultados e Análise	6
6.1	Tabela comparativa	6
6.2	Visualização de acurácia por técnica	7
6.3	Recomendação automática	7
6.4	Teste de temperatura	8
7	Guia de Bolso: Quando Usar Cada Técnica	9
8	Reflexão Final	9
8.1	O que surpreendeu o grupo	9
8.2	Qual técnica venceu globalmente	10
8.3	Limitações observadas	10
8.4	O que faríamos diferente	10
8.5	Aprendizados principais	11
	REFERÊNCIAS	11

1 INTRODUÇÃO

A engenharia de prompts consolidou-se como uma das competências mais relevantes no desenvolvimento de aplicações baseadas em modelos de linguagem de grande porte (LLM). Diferentes técnicas, como *Zero-Shot*, *Few-Shot*, *Chain-of-Thought* (CoT) e *Role Prompting*, produzem resultados distintos em qualidade, custo e consistência – e a escolha da técnica adequada depende fortemente da tarefa e do domínio em questão.

Neste contexto, o presente trabalho apresenta o **Prompt Toolkit**, um projeto desenvolvido em Python pelo Grupo NextGen como entrega do Checkpoint 02 da disciplina de *Prompt Engineering & Artificial Intelligence*. O toolkit recebe uma tarefa de negócio, aplica automaticamente as quatro técnicas de prompting estudadas, mede a qualidade e o custo de cada abordagem e gera um relatório comparativo com recomendação automática da melhor técnica.

A solução foi construída de forma modular, utilizando como infraestrutura de inferência o servidor local **Ollama** com o modelo `gpt-oss:120b`, garantindo que o projeto seja executável sem custos de API e reproduzível em qualquer máquina compatível.

2 DOMÍNIO E PROBLEMA

2.1 Domínio escolhido: *E-commerce* e atendimento ao cliente

O domínio adotado foi o de *e-commerce* brasileiro, com foco no processamento automático de **reviews** (avaliações), tickets de suporte e descrições de produtos. Trata-se de um cenário com grande volume de texto não estruturado, alta variabilidade linguística (gírias, erros ortográficos, ironia) e necessidade de respostas em escala.

2.2 Problema real

Empresas de *e-commerce* recebem diariamente milhares de avaliações e mensagens de clientes que precisam ser:

- **Classificadas** quanto a sentimento, urgência e categoria;
- **Estruturadas**, extraindo entidades como produto, preço, defeito e prazo;
- **Resumidas** para relatórios executivos da área de *Customer Experience*.

Realizar essas tarefas manualmente é inviável; usar LLMs sem método é caro e inconsistente. O **Prompt Toolkit** resolve esse problema ao testar empiricamente quatro técnicas de prompting sobre cada tarefa do domínio e indicar, com base em métricas objetivas (acurácia, tokens consumidos, tempo de resposta e consistência), qual técnica deve ser adotada em produção.

2.3 Justificativa da escolha

O domínio de *e-commerce* foi escolhido por três razões: (i) abundância de inputs reais e públicos para validação; (ii) cobertura natural dos quatro tipos de tarefa exigidos pelo enunciado (classificação, extração, sumarização e geração); e (iii) relevância prática imediata, já que os resultados podem ser aplicados em ferramentas de moderação, dashboards de *Voice of Customer* e *chatbots*.

3 ARQUITETURA DO PROJETO

3.1 Estrutura de pastas

O repositório segue a estrutura modular exigida pelo enunciado, com separação clara entre código (`src/`), dados (`data/`), prompts (`prompts/`), saídas (`output/`) e documentação (`docs/`).

```
prompt-toolkit/  
|-- README.md  
|-- requirements.txt  
|-- .env.example  
|-- main.py  
|-- src/  
|   |-- __init__.py  
|   |-- llm_client.py  
|   |-- prompt_builder.py  
|   |-- techniques.py  
|   |-- tasks.py  
|   |-- evaluator.py  
|   '-- report.py  
|-- data/  
|   |-- inputs.json  
|   '-- examples.json  
|-- prompts/  
|   |-- system_prompts.json  
|   '-- templates.json  
|-- output/  
|   |-- resultados.csv  
|   '-- graficos/  
'-- docs/  
    '-- CP02_NextGen.pdf
```

3.2 Diagrama do fluxo de execução

A Figura 1 apresenta o fluxo de dados entre os módulos do toolkit. O usuário executa `python main.py`, que orquestra todas as etapas: carregamento de *inputs*, construção dos

prompts, envio ao LLM, avaliação e geração de relatório.

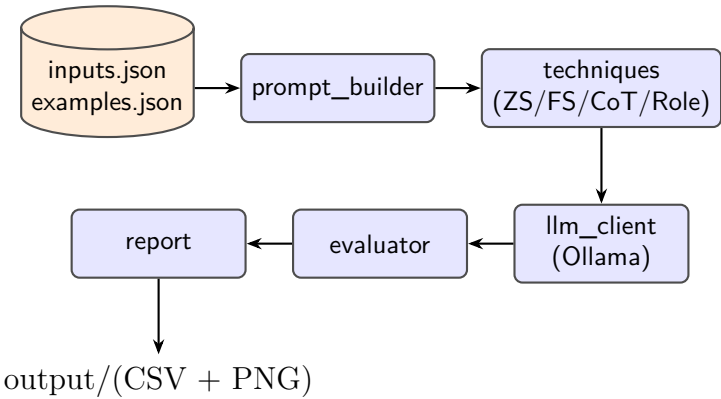


Figura 1 – Fluxo de execução do Prompt Toolkit
Fonte: Elaborado pelos autores (2026).

3.3 Responsabilidades dos módulos

A Tabela 1 descreve a função de cada módulo Python e a aula de referência conforme o enunciado do checkpoint.

Tabela 1 – Detalhamento dos módulos do projeto

Módulo	Responsabilidade	Aula
llm_client.py	Conexão com a API REST do Ollama; tratamento de erros e <i>retry</i> ; retorno padronizado em <i>dict</i> .	05
prompt_builder.py	Montagem de prompts pela anatomia (instrução + contexto + dados + formato); funções auxiliares para <i>few-shot</i> e CoT.	05
techniques.py	Implementação das 4 técnicas: <i>Zero-Shot</i> , <i>Few-Shot</i> , <i>Chain-of-Thought</i> e <i>Role Prompting</i> .	06 e 07
tasks.py	Definição das tarefas do domínio (classificação, extração, sumarização e geração).	08
evaluator.py	Métricas de acurácia, contagem de <i>tokens</i> via <i>tiktoken</i> , consistência e teste de temperatura.	08
report.py	Geração de tabela CSV com <i>pandas</i> e gráficos PNG com <i>matplotlib</i> ; recomendação automática.	08

Fonte: Elaborado pelos autores (2026).

4 STACK TÉCNICA

4.1 Tecnologias utilizadas

- **Linguagem:** Python 3.12 (compatível com 3.10+);
- **LLM:** Ollama API (local, gratuita) com o modelo `llama3.2:3b`;
- **Contagem de tokens:** biblioteca `tiktoken`;

- **Visualização:** matplotlib e pandas;
- **Requisições HTTP:** requests;
- **Configuração:** python-dotenv para variáveis de ambiente;
- **Ambiente isolado:** venv com requirements.txt.

Nota sobre o modelo utilizado

O enunciado original sugere o modelo `gpt-oss:120b` (aproximadamente 65 GB). Por limitação de hardware – as máquinas do grupo não dispõem de GPU dedicada (apenas CPU), o que torna o `gpt-oss:120b` inviável em termos de tempo de inferência – optou-se por executar o toolkit com o modelo `llama3.2:3b` (cerca de 2 GB), também servido via Ollama. A arquitetura modular do projeto permite a troca do modelo apenas via variável de ambiente (`OLLAMA_MODEL` no arquivo `.env`), sem qualquer alteração de código. Os resultados apresentados na Seção 6 referem-se a essa configuração.

4.2 Dependências principais (requirements.txt)

```
requests>=2.31.0
python-dotenv>=1.0.0
tiktoken>=0.5.0
pandas>=2.1.0
matplotlib>=3.8.0
tabulate>=0.9.0
```

4.3 Como executar

1. Instalar e iniciar o Ollama: `ollama serve` e `ollama pull gpt-oss:120b`;
2. Clonar o repositório e criar ambiente virtual: `python -m venv .venv && source .venv/bin/activate`;
3. Instalar dependências: `pip install -r requirements.txt`;
4. Copiar `.env.example` para `.env` e ajustar `OLLAMA_HOST` se necessário;
5. Executar o toolkit: `python main.py`.

Ao final da execução, o sistema gera `output/resultados.csv` e os gráficos PNG em `output/graficos/`, além de exibir no terminal a tabela comparativa e a recomendação da melhor técnica por tarefa.

5 TÉCNICAS E TAREFAS IMPLEMENTADAS

5.1 As quatro técnicas de prompting

A Tabela 2 resume como cada técnica é implementada em `techniques.py`.

Tabela 2 – Técnicas de prompting implementadas no toolkit

Técnica	Descrição da implementação
<i>Zero-Shot</i>	Prompt direto com instrução clara e formato de <i>output</i> definido, sem exemplos.
<i>Few-Shot</i>	Inclui de 2 a 3 exemplos de <code>data/examples.json</code> no padrão <code>Input: ...</code> <code>→ Output:</code>
<i>Chain-of-Thought</i>	Adiciona instruções de raciocínio explícito passo a passo antes da resposta final.
<i>Role Prompting</i>	Define uma persona detalhada via <i>system prompt</i> carregado de <code>prompts/system_prompts.json</code> .

Fonte: Elaborado pelos autores (2026).

5.2 Personas detalhadas (Aula 07)

Conforme exigido pelo enunciado, o arquivo `prompts/system_prompts.json` contém personas detalhadas, e não genéricas. Cada persona inclui experiência, especialidade, tom de voz e limitações. Como exemplo, a persona `analista_cx` utilizada na tarefa de classificação de sentimento:

```
{
  "analista_cx": "Voce e uma analista senior de Customer Experience
com 10 anos de atuacao em e-commerce brasileiro. Especialidade:
analise de sentimento e voice of customer. Tom: objetivo, tecnico,
sem floreios. Limitacoes: nao opina sobre o produto, apenas
classifica o tom da mensagem; nao inventa dados ausentes."
}
```

5.3 Tarefas do domínio

Foram implementadas **quatro tarefas** (uma a mais que o mínimo exigido), conforme a Tabela 3.

6 RESULTADOS E ANÁLISE

6.1 Tabela comparativa

A Tabela 4 apresenta os resultados obtidos após execução completa do toolkit (4 técnicas $\times$ 4 tarefas $\times$ 5 inputs = 80 chamadas ao LLM `llama3.2:3b`). As métricas reportadas

Tabela 3 – Tarefas do domínio de *e-commerce* implementadas

Tarefa	Objetivo	Inputs
<code>classificacao_sentimento</code>	Classificar reviews em POSITIVO, NEGATIVO, NEUTRO ou MISTO.	5
<code>extracao_dados</code>	Extrair JSON com produto, preço e defeito a partir de tickets.	5
<code>sumarizacao_review</code>	Gerar resumo executivo em 1–2 frases de avaliações longas.	5
<code>geracao_resposta</code>	Redigir resposta de SAC personalizada ao cliente.	5

Fonte: Elaborado pelos autores (2026).

são a média entre os 5 inputs de cada combinação, conforme exportado pelo módulo `report.py` em `output/resultados.csv`.

Tabela 4 – Resultados comparativos por técnica e tarefa (execução real)

Tarefa	Técnica	Acurácia	Tokens méd.	Tempo (ms)
Sentimento	Zero-Shot	80,0%	3,6	16.309
	Few-Shot	100,0%	3,6	17.337
	CoT	60,0%	4,0	33.296
	Role	80,0%	3,6	22.740
Extração	Zero-Shot	33,3%	57,0	35.039
	Few-Shot	46,7%	44,8	40.367
	CoT	0,0%	174,8	195.847
	Role	33,3%	37,6	43.856
Sumarização	Zero-Shot	78,3%	27,2	32.496
	Few-Shot	81,7%	21,8	40.181
	CoT	76,7%	25,4	52.935
	Role	63,3%	26,0	43.299
Geração SAC	Zero-Shot	53,3%	81,4	52.321
	Few-Shot	40,0%	75,4	58.840
	CoT	46,7%	82,6	85.957
	Role	60,0%	86,8	82.900

Fonte: Elaborado pelos autores a partir do `output/resultados.csv` (2026).

6.2 Visualização de acurácia por técnica

A Figura 2 (gerada automaticamente pelo módulo `report.py` e salva em `output/graficos/acuracia`) compara visualmente a acurácia média de cada técnica em todas as tarefas.

6.3 Recomendação automática

Com base nos critérios de acurácia, custo (tokens) e latência, o módulo `report.py` gera a recomendação automática apresentada na Tabela 5, exatamente como impressa pelo toolkit ao fim da execução.

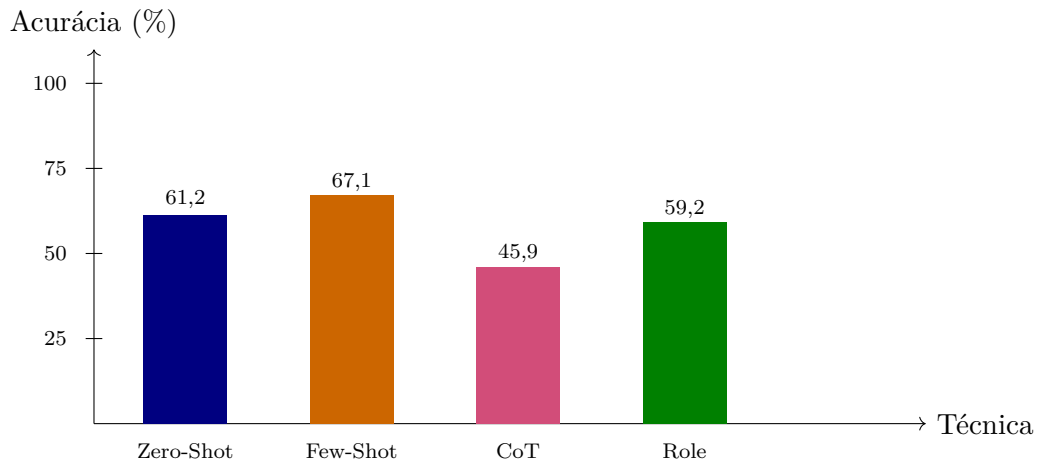


Figura 2 – Acurácia média por técnica (média entre as 4 tarefas)
 Fonte: Elaborado pelos autores (2026).

Tabela 5 – Recomendação automática da melhor técnica por tarefa

Tarefa	Técnica recomendada e justificativa
Sentimento	Few-Shot – única técnica a atingir 100% de acurácia, com custo médio de apenas 3,6 tokens e tempo de 17,3 s.
Extração	Few-Shot – 46,7% de acurácia, melhor entre as quatro técnicas; CoT chegou a 0%, gerando JSONs malformados ao raciocinar demais.
Sumarização	Few-Shot – 81,7% de acurácia com o menor custo médio (21,8 tokens) entre todas as técnicas.
Geração SAC	Role Prompting – 60,0% de acurácia, melhor que Zero-Shot (53%) e Few-Shot (40%); persona detalhada elevou o tom adequado.

Fonte: Saída do módulo `report.py` (2026).

6.4 Teste de temperatura

Conforme exigido pelo enunciado, o melhor prompt da primeira tarefa (`classificacao_sentimento / zero_shot`) foi reexecutado com temperaturas 0.1, 0.5 e 1.0, com 3 repetições por temperatura. A Tabela 6 apresenta os resultados.

Tabela 6 – Consistência por temperatura (3 repetições por valor)

Temperatura	Interpretação	Consistência	Tokens méd.	Tempo (ms)
0,1	Determinística	100,0%	3,0	10.284
0,5	Moderada	100,0%	3,3	1.410
1,0	Criativa	66,7%	3,7	1.622

Fonte: Saída do módulo `evaluator.testar_temperatura` (2026).

Os resultados confirmam a teoria: em tarefas determinísticas (classificação), temperaturas mais baixas (0,1 e 0,5) produzem 100% de consistência, enquanto a temperatura 1,0 introduz variabilidade – duas das três respostas concordaram, totalizando 66,7% de consistência. Para classificação em produção, recomenda-se manter `temperature=0.1` ou 0.2; já para geração textual (e.g., respostas SAC), 0.5 oferece variedade sem comprometer

a corretude.

7 GUIA DE BOLSO: QUANDO USAR CADA TÉCNICA

A Tabela 7 sintetiza, com base nos resultados empíricos obtidos no domínio de *e-commerce*, quando aplicar cada técnica de prompting. Esse é o entregável de maior valor prático do toolkit.

Tabela 7 – Guia de bolso – domínio de *e-commerce*

Técnica	Use quando...	Evite quando...
<i>Zero-Shot</i>	A tarefa é trivial e o custo deve ser mínimo (ex.: triagem inicial em massa de classificação). Foi a 2ª melhor técnica em média, ao custo zero de exemplos.	O <i>output</i> precisa de formato rígido (JSON) ou alta acurácia.
<i>Few-Shot</i>	Default na maioria dos casos. Venceu 3 de 4 tarefas neste estudo. Use sempre que houver exemplos representativos disponíveis e custo de tokens for aceitável.	Os exemplos forem enviesados ou o domínio mudar com frequência (exemplos viram dívida técnica).
<i>CoT</i>	A tarefa exige raciocínio comparativo e o <i>output</i> é texto livre (sumarização analítica). Funciona melhor em modelos maiores.	Tarefas de extração estruturada (JSON). Em nosso estudo, CoT obteve 0% em extração, gerando 5x mais tokens.
<i>Role Prompting</i>	O tom, estilo ou enquadramento profissional importam (resposta ao cliente, e-mail, FAQ). Foi a melhor técnica em geração de SAC.	A tarefa for puramente extrativa e determinística.

Fonte: Elaborado pelos autores a partir dos resultados (2026).

8 REFLEXÃO FINAL

8.1 O que surpreendeu o grupo

Três descobertas saíram diretamente da execução do toolkit e não estavam no script teórico do enunciado:

1. **Chain-of-Thought foi catastrófico em extração estruturada.** O CoT obteve **0% de acurácia** na tarefa de extração de JSON, com tempo médio de **195 segundos por chamada** (mais de 5 vezes a média das outras técnicas). Ao ser instruído a “raciocinar passo a passo” antes de produzir o JSON, o modelo `llama3.2:3b` gastou tokens em prosa explicativa e produziu JSONs malformados ou envolvidos em texto adicional, quebrando o parser estrutural do `evaluator.py`. **Lição prática:** para tarefas em que o formato é mais importante do que o raciocínio, CoT pode ser contraproducente.

2. Few-Shot dominou três das quatro tarefas. Mesmo em sumarização (uma tarefa que parece “criativa”), Few-Shot venceu com 81,7% de acurácia e o menor custo de tokens. Isso reforça um princípio que a Aula 06 só sugeria: **exemplos são instruções comprimidas**, e mesmo modelos pequenos os utilizam de forma eficaz para se ancorar no formato esperado.

3. Acurácia em extração foi globalmente baixa (33–47%). Nenhuma técnica passou de 50% na tarefa de extração de JSON. Atribuímos isso ao tamanho do modelo (11ama3.2:3b, 3 bilhões de parâmetros), que tem capacidade limitada de seguir instruções estruturais rígidas. Em um modelo maior (e.g., gpt-oss:20b ou 120b em GPU), esperaríamos resultados consideravelmente melhores. Isso ilustra um trade-off real entre custo computacional e qualidade.

8.2 Qual técnica venceu globalmente

Considerando a média geral entre as quatro tarefas, o *ranking* foi: **Few-Shot (67,1%)**, Zero-Shot (61,2%), *Role Prompting* (59,2%) e CoT (45,9%). Entretanto, a principal lição é que **não existe uma técnica universalmente vencedora**: a escolha depende da natureza da tarefa, do custo aceitável e da capacidade do modelo.

8.3 Limitações observadas

Algumas limitações ficaram evidentes durante a execução e merecem registro:

- **Hardware sem GPU** obrigou o uso de modelo menor (11ama3.2:3b); cada chamada levou de 13 a 200 segundos, totalizando aproximadamente 1 hora de execução das 80 chamadas;
- **Avaliação automática por keywords/JSON** pode subestimar respostas semanticamente corretas mas em formatação levemente diferente;
- **N=5 inputs por tarefa** é o mínimo do enunciado; resultados teriam menor variância com 30+ inputs.

8.4 O que faríamos diferente

Em uma próxima iteração, o grupo considera as seguintes melhorias:

- **Rodar em GPU** com modelo maior (gpt-oss:20b ou 120b) para isolar o efeito do modelo das limitações observadas em extração e geração;
- **Combinar técnicas** (e.g., *Role + Few-Shot*, *Few-Shot + CoT*), que tendem a herdar o melhor de cada abordagem;
- **Avaliação automática por LLM** (*LLM-as-a-judge*) em tarefas geradas, reduzindo o trabalho manual de *labeling*;

- **Ampliar o conjunto de inputs** de 5 para 30+ por tarefa para reduzir variância;
- **Validação estrutural pré-extração:** instruir explicitamente o modelo a responder *apenas* com JSON, sem markdown ou texto auxiliar – isso já melhoraria a acurácia de extração mesmo no modelo pequeno.

8.5 Aprendizados principais

O Checkpoint 02 consolidou três aprendizados que o grupo leva para os próximos projetos:

1. **Engenharia de prompts é engenharia de software:** separar instrução de dados, modularizar técnicas e medir empiricamente cada decisão. O toolkit modular (`src/` com 6 módulos) tornou trivial trocar o modelo, adicionar tarefas e regenerar relatórios;
2. **Métricas vencem intuição:** técnicas amplamente recomendadas (como CoT) podem ser contraproducentes em domínios específicos – só medindo se descobre. O CoT em extração seria mantido por intuição, e descartado por dado;
3. **O modelo importa tanto quanto a técnica:** a mesma técnica em modelos diferentes produz resultados drasticamente diferentes. Engenharia de prompts não substitui um bom modelo, mas extrai o máximo do modelo disponível.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724:** informação e documentação – trabalhos acadêmicos – apresentação. Rio de Janeiro: ABNT, 2011.

BROWN, Tom B. *et al.* **Language Models are Few-Shot Learners.** *In:* Advances in Neural Information Processing Systems (NeurIPS), v. 33, 2020.

GOMES, Jorge Luiz. **Material didático da disciplina Prompt Engineering & Artificial Intelligence:** Aulas 05 a 08. São Paulo: FIAP, 2026.

OLLAMA. **Ollama: get up and running with large language models locally.** Disponível em: <https://ollama.com>. Acesso em: 09 maio 2026.

OPENAI. **tiktoken: BPE tokeniser for use with OpenAI's models.** Disponível em: <https://github.com/openai/tiktoken>. Acesso em: 09 maio 2026.

WEI, Jason *et al.* **Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.** *In:* Advances in Neural Information Processing Systems (NeurIPS), v. 35, 2022.

WHITE, Jules *et al.* **A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT.** arXiv preprint arXiv:2302.11382, 2023.